

Android Deep Dive And Build Manual

- Android Deep-Dive & Build Manual
 - 1. Architecture Overview
 - 1.1 High-Level Structure
 - 1.2 Core Modules (`core/`)
 - 1.3 UI Layer (`ui/`)
 - 2. Cognito Custom Auth Integration
 - 2.1 Wire Format (Backend Contract)
 - 2.2 `NetworkPeerApi.kt` — Retrofit Interface
 - 2.3 Auth Interceptor & Token Refresh (OkHttp)
 - 2.4 `NetworkPeerRepository` — Token Management
 - 3. Local Development & Physical Device Testing
 - 3.1 Prerequisites
 - 3.2 Environment Setup
 - 3.3 Local Properties (SDK Path)
 - 3.4 Network Security Config (Local Backend)
 - 3.5 Build Variants (`build.gradle.kts`)
 - 3.6 Build & Run on Physical Device
 - 3.7 Run Unit Tests
 - 3.8 Emulator (Alternative)
 - 4. Evidence Upload Pipeline (S3 Presigned)
 - 4.1 Flow (Identical to iOS)
 - 4.2 Implementation (Kotlin + OkHttp)
 - 5. Push Notifications (FCM + AWS SNS)
 - 5.1 Firebase Setup
 - 5.2 FCM Integration
 - 5.3 `AndroidManifest` Permissions
 - 5.4 SNS Integration (Backend)
 - 6. Release Signing & Google Play Deployment
 - 6.1 Keystore Generation
 - 6.2 Signing Config (`app/build.gradle.kts`)

- [6.3 ProGuard/R8 Rules \(proguard-rules.pro\)](#)
- [6.4 Build AAB for Play Store](#)
- [6.5 Play Console Setup](#)
- [7. Testing](#)
 - [7.1 Unit Tests \(JUnit + Kotlinx Serialization\)](#)
 - [7.2 Contract Tests](#)
 - [7.3 Instrumented Tests \(AndroidJUnitRunner\)](#)
- [8. CI/CD \(GitHub Actions\)](#)
 - [8.1 Workflow \(`.github/workflows/android.yml` \)](#)
- [9. Troubleshooting](#)
- [10. Quick Reference Commands](#)

Android Deep-Dive & Build Manual

NetworkPeer Android Application — Native Kotlin / Jetpack Compose / Gradle KTS

1. Architecture Overview

1.1 High-Level Structure

```

apps/android/
├─ build.gradle.kts           # Root Gradle config (plugins, repositories)
├─ settings.gradle.kts       # Module inclusion
├─ gradle.properties         # JVM args, version constants
├─ gradle/                   # Gradle wrapper + version catalogs
├─ app/
|   ├─ build.gradle.kts      # App module: deps, signing, variants
|   ├─ src/
|   |   ├─ main/
|   |   |   ├─ AndroidManifest.xml
|   |   |   ├─ java/com/networkpeer/mobile/
|   |   |   |   ├─ core/      # Business logic (DI, data, network, model)
|   |   |   |   |   ├─ data/  # Repository, DataStore, Database
|   |   |   |   |   ├─ model/ # Data classes (kotlinx.serialization)
|   |   |   |   |   ├─ network/ # Retrofit + Socket.IO clients
|   |   |   |   |   └─ notifications/ # FCM + Push handling
|   |   |   |   └─ di/       # Hilt modules
|   |   |   └─ ui/          # Compose screens, ViewModels, Navigation
|   |   └─ NetworkPeerApp.kt # Application class (Hilt entry)
|   └─ res/                 # Resources (themes, drawables, xml)
|       └─ assets/
|   └─ test/                # Unit tests (JUnit + Kotlinx serialization)
|       └─ androidTest/     # Instrumented tests
├─ networkpeer.development.local.properties.example
├─ networkpeer.production.local.properties.example
├─ gradlew / gradlew.bat    # Gradle wrapper
└─ local.properties         # SDK path (gitignored)

```

1.2 Core Modules (core/)

Package	Responsibility
data/	<code>NetworkPeerRepository</code> (single source of truth), <code>PreferencesDataStore</code> (tokens), <code>AppDatabase</code> (Room - optional cache)
model/	<code>@Serializable</code> data classes matching backend OpenAPI; <code>snake_case</code> JSON via <code>@SerializedName</code>
network/	<code>NetworkPeerApi</code> (Retrofit interface), <code>NetworkPeerService</code> (Retrofit + OkHttp + Auth interceptor), <code>SocketService</code> (Socket.IO)
notifications/	<code>FcmIntegration</code> (FCM token + message handling), <code>PushNotificationService</code> (foreground/background)
di/	Hilt modules: <code>NetworkModule</code> , <code>RepositoryModule</code> , <code>DatabaseModule</code>

1.3 UI Layer (ui/)

- **Framework:** Jetpack Compose (Material 3)
 - **Architecture:** MVVM with `ViewModel` + `StateFlow` / `MutableStateFlow`
 - **Navigation:** Navigation Compose (type-safe routes via `@Serializable` params)
 - **Dependency Injection:** Hilt (`@HiltAndroidApp` , `@AndroidEntryPoint`)
-

2. Cognito Custom Auth Integration

2.1 Wire Format (Backend Contract)

```

// Request OTP
@Serializable
data class OTPRequest(
    @SerializedName("phone_number") val phoneNumber: String,
    val role: UserRole // CLIENT | WORKER
)

// Response (snake_case from backend)
@Serializable
data class OTPResponse(
    @SerializedName("challenge_id") val challengeId: String,
    @SerializedName("expires_in_seconds") val expiresInSeconds: Int,
    @SerializedName("otp_length") val otpLength: Int,
    val delivery: DeliveryInfo
)

@Serializable
data class DeliveryInfo(
    val transport: String, // "sms"
    val to: String
)

// Verify OTP
@Serializable
data class OTPVerifyRequest(
    @SerializedName("phone_number") val phoneNumber: String,
    @SerializedName("challenge_id") val challengeId: String,
    val otp: String,
    val transport: String // "sms" | "browser"
)

// Token Response
@Serializable
data class TokenResponse(
    @SerializedName("access_token") val accessToken: String,
    @SerializedName("refresh_token") val refreshToken: String,
    @SerializedName("id_token") val idToken: String,
    @SerializedName("expires_in") val expiresIn: Int,

```

```
val user: UserProfile  
)
```

2.2 NetworkPeerApi.kt — Retrofit Interface


```

interface NetworkPeerApi {
    // Auth
    @POST("auth/otp/request")
    suspend fun requestOtp(@Body request: OTPRequest): OTPResponse

    @POST("auth/otp/verify")
    suspend fun verifyOtp(@Body request: OTPVerifyRequest): TokenResponse

    @POST("auth/token/refresh")
    suspend fun refreshToken(@Body request: RefreshTokenRequest): TokenResponse

    @POST("auth/logout")
    suspend fun logout(@Body request: LogoutRequest): Unit

    // Jobs
    @GET("jobs")
    suspend fun listJobs(@Query("cursor") cursor: String?, @QueryMap filters:
        Map<String, String>): PaginatedResponse<Job>

    @POST("jobs")
    suspend fun createJob(@Body request: CreateJobRequest): Job

    @POST("jobs/{id}/accept")
    suspend fun acceptJob(@Path("id") jobId: String): Job

    @POST("jobs/{id}/complete")
    suspend fun completeJob(@Path("id") jobId: String): Job

    // Evidence (3-step S3)
    @POST("evidence/reserve")
    suspend fun reserveEvidenceUpload(@Body request: EvidenceReserveRequest):
        EvidenceReservation

    @POST("evidence/complete")
    suspend fun completeEvidenceUpload(@Body request:
        EvidenceCompleteRequest): Evidence

    // Notifications
    @POST("notifications/device-token")
    suspend fun registerDeviceToken(@Body request: DeviceTokenRequest): Unit

```

```
@DELETE("notifications/device-token")
suspend fun unregisterDeviceToken(@Body request: DeviceTokenRequest): Unit

// Socket.IO (via separate SocketService)
companion object {
    const val BASE_URL = "https://api.networkpeer.com/" // Override via
    BuildConfig
}
}
```

2.3 Auth Interceptor & Token Refresh (OkHttp)

```
class AuthInterceptor @Inject constructor(  
    private val repository: NetworkPeerRepository  
) : Interceptor {  
    override fun intercept(chain: Interceptor.Chain): Response {  
        val original = chain.request()  
        val accessToken = repository.getAccessToken() ?: return  
            chain.proceed(original)  
  
        val authenticated = original.newBuilder()  
            .header("Authorization", "Bearer $accessToken")  
            .build()  
  
        var response = chain.proceed(authenticated)  
  
        if (response.code == 401) {  
            // Attempt refresh  
            val newToken = repository.refreshAccessToken()  
            if (newToken != null) {  
                val retry = original.newBuilder()  
                    .header("Authorization", "Bearer $newToken")  
                    .build()  
                response = chain.proceed(retry)  
            }  
        }  
        return response  
    }  
}
```

2.4 NetworkPeerRepository — Token Management

```

@Singleton
class NetworkPeerRepository @Inject constructor(
    private val api: NetworkPeerApi,
    private val datastore: PreferencesDataStore,
    private val fcmIntegration: FcmIntegration
) {
    // DataStore keys
    private val ACCESS_TOKEN_KEY = stringPreferencesKey("access_token")
    private val REFRESH_TOKEN_KEY = stringPreferencesKey("refresh_token")
    private val USER_PROFILE_KEY = stringPreferencesKey("user_profile")

    // In-memory access token (short-lived)
    private var _accessToken: String? = null
    val accessToken: String? get() = _accessToken

    suspend fun requestOtp(phoneNumber: String, role: UserRole): OTPResponse {
        return api.requestOtp(OTPRequest(phoneNumber, role))
    }

    suspend fun verifyOtp(challengeId: String, otp: String, transport:
        String): TokenResponse {
        val response = api.verifyOtp(OTPVerifyRequest(
            phoneNumber = currentPhoneNumber!!,
            challengeId = challengeId,
            otp = otp,
            transport = transport
        ))
        saveTokens(response)
        return response
    }

    private fun saveTokens(response: TokenResponse) {
        _accessToken = response.accessToken
        datastore.edit { it[REFRESH_TOKEN_KEY] = response.refreshToken }
        datastore.edit { it[USER_PROFILE_KEY] =
            Json.encodeToString(response.user) }
    }

    suspend fun refreshAccessToken(): String? {

```

```
val refreshToken = datastore.data.first()[REFRESH_TOKEN_KEY] ?: return
    null
val response = api.refreshToken(RefreshTokenRequest(refreshToken))
_accessToken = response.accessToken
datastore.edit { it[REFRESH_TOKEN_KEY] = response.refreshToken }
return response.accessToken
}

suspend fun logout() {
    val refreshToken = datastore.data.first()[REFRESH_TOKEN_KEY] ?: return
    api.logout(LoginRequest(refreshToken))
    clearTokens()
}

private fun clearTokens() {
    _accessToken = null
    datastore.edit { it[REFRESH_TOKEN_KEY] = "" }
    datastore.edit { it[USER_PROFILE_KEY] = "" }
}
}
```

3. Local Development & Physical Device Testing

3.1 Prerequisites

Tool	Version	Install
Android Studio	Ladybug (2024.2)+	developer.android.com
JDK	17 (Temurin/OpenJDK)	<code>brew install openjdk@17</code>
Android SDK	API 34 (Android 14)	Android Studio SDK Manager
Physical Device	Android 8.0+ (API 26+)	USB Debugging enabled

3.2 Environment Setup

```
# Add to ~/.zshrc or ~/.bashrc
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/
openjdk.jdk/Contents/Home"

export ANDROID_HOME="/opt/homebrew/share/android-commandlinetools"
export PATH="$ANDROID_HOME/cmdline-tools/latest/bin:$ANDROID_HOME/platform-
tools:$PATH"

# Verify
java -version
adb version
```

3.3 Local Properties (SDK Path)

Create `local.properties` in `apps/android/` :

```
sdk.dir=/opt/homebrew/share/android-commandlinetools
```

3.4 Network Security Config (Local Backend)

`res/xml/network_security_config.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <domain-config cleartextTrafficPermitted="true">
    <domain includeSubdomains="true">10.0.2.2</domain> <!-- Android
emulator localhost -->
    <domain includeSubdomains="true">192.168.1.XXX</domain> <!-- Your Mac
LAN IP -->
    <domain includeSubdomains="true">localhost</domain>
  </domain-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system"/>
    </trust-anchors>
  </base-config>
</network-security-config>
```

`AndroidManifest.xml`:

```
<application
    android:networkSecurityConfig="@xml/network_security_config"
    ... >
```

3.5 Build Variants (build.gradle.kts)

```
buildTypes {
    debug {
        isDebuggable = true
        isMinifyEnabled = false
        applicationIdSuffix = ".debug"
        versionNameSuffix = "-dev"
        buildConfigField("String", "BASE_URL", "\"http://192.168.1.XXX:3000/\"")
    }
    release {
        isMinifyEnabled = true
        proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro")
        signingConfig = signingConfigs.getByName("release")
        buildConfigField("String", "BASE_URL", "\"https://api.networkpeer.com/\"")
    }
}

productFlavors {
    development {
        dimension = "environment"
        applicationIdSuffix = ".dev"
    }
    production {
        dimension = "environment"
    }
}
```


3.6 Build & Run on Physical Device

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/android

# 1. Enable USB Debugging on phone:
#   Settings → About Phone → Tap Build Number 7x → Developer Options → USB
#   Debugging

# 2. Connect via USB → Allow debugging on phone

# 3. Verify connection
adb devices
# Should show: <serial>    device

# 4. Build & install debug variant
./gradlew installDevelopmentDebug

# Or build APK only
./gradlew assembleDevelopmentDebug
# Output: app/build/outputs/apk/development/debug/app-development-debug.apk

# 5. Install manually (if needed)
adb install -r app/build/outputs/apk/development/debug/app-development-
debug.apk
```

3.7 Run Unit Tests

```
# All unit tests
./gradlew test

# Specific variant
./gradlew testDevelopmentDebugUnitTest

# With coverage
./gradlew jacocoTestReport
# Report: app/build/reports/jacoco/testDevelopmentDebugUnitTest/html/index.html
```

3.8 Emulator (Alternative)

```
# Create AVD
avdmanager create avd -n pixel8 -k "system-images;android-34;google_apis;arm64"

# Start emulator
emulator -avd pixel8

# Run on emulator
./gradlew installDevelopmentDebug
```

4. Evidence Upload Pipeline (S3 Presigned)

4.1 Flow (Identical to iOS)

1. POST /evidence/reserve → { reservation_id, upload: { url, fields } }
2. PUT direct to S3 (multipart/form-data)
3. POST /evidence/complete → Evidence record

4.2 Implementation (Kotlin + OkHttp)

```

suspend fun uploadEvidence(
    jobId: String,
    subtaskId: String?,
    mediaType: MediaType,
    mimeType: String,
    file: File,
    capturedAt: Instant
): Evidence {
    // Step 1: Reserve
    val reservation = api.reserveEvidenceUpload(
        EvidenceReserveRequest(
            jobId = jobId,
            subtaskId = subtaskId,
            mediaType = mediaType,
            mimeType = mimeType,
            fileSizeBytes = file.length()
        )
    )

    // Step 2: Multipart upload to S3
    val requestBody = MultipartBody.Builder()
        .setType(MultipartBody.FORM)
        .apply {
            reservation.fields.forEach { (key, value) ->
                addFormDataPart(key, value)
            }
            addFormDataPart(
                "file",
                file.name,
                file.asRequestBody(mimeType.toMediaTypeOrNull() ?:
                    MediaType.parse("application/octet-stream"))
            )
        }
        .build()

    val s3Request = Request.Builder()
        .url(reservation.uploadUrl)
        .post(requestBody)
        .build()

```

```
okHttpClient.newCall(s3Request).execute().use { response ->
    if (!response.isSuccessful) throw IOException("S3 upload failed: $
        {response.code}")
}

// Step 3: Complete
return api.completeEvidenceUpload(
    EvidenceCompleteRequest(
        reservationId = reservation.id,
        s3Key = reservation.s3Key,
        capturedAt = capturedAt
    )
)
```

5. Push Notifications (FCM + AWS SNS)

5.1 Firebase Setup

1. **Firebase Console** → Create project → Add Android app
2. **Package name:** `com.networkpeer.mobile` (must match `applicationId`)
3. Download `google-services.json` → `app/`
4. Enable **Cloud Messaging** API

5.2 FCM Integration

```
// FcmIntegration.kt
class FcmIntegration @Inject constructor(
    @ApplicationContext context: Context
) {
    private val firebaseMessaging = FirebaseMessaging.getInstance()

    suspend fun initialize() {
        val token = firebaseMessaging.getToken().await()
        repository.registerDeviceToken(token, "FCM")
    }

    // Handle foreground messages
    fun onMessageReceived(remoteMessage: RemoteMessage) {
        // Parse DataOnlyPushPayload → Route to appropriate handler
    }
}

// Register in NetworkPeerApp.onCreate()
hiltApplication.onCreate { fcmIntegration.initialize() }
```

5.3 AndroidManifest Permissions

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="com.google.android.c2dm.permission.RECEIVE" />

<service
    android:name=".core.notifications.FcmMessageService"
    android:exported="true">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

5.4 SNS Integration (Backend)

- Backend stores FCM tokens in `device_tokens` table (`platform = 'FCM'`)
 - On push: BullMQ worker → `firebase-admin` SDK → `messaging.send()`
 - Payload includes `data` for silent sync + `notification` for user-visible alert
-

6. Release Signing & Google Play Deployment

6.1 Keystore Generation

```
# Generate release keystore (run ONCE, store securely!)
keytool -genkeypair -v \
  -keystore release.keystore \
  -alias networkpeer \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -storepass <STORE_PASSWORD> \
  -keypass <KEY_PASSWORD> \
  -dname "CN=NetworkPeer, OU=Engineering, O=NetworkPeer, L=City, ST=State, C=US"

# Store passwords in ~/.gradle/gradle.properties (never commit!)
# networkpeer.keystore.password=<STORE_PASSWORD>
# networkpeer.key.password=<KEY_PASSWORD>
```

6.2 Signing Config (app/build.gradle.kts)

```
signingConfigs {  
    create("release") {  
        val keystoreFile = rootProject.file("../release.keystore")  
        if (keystoreFile.exists()) {  
            storeFile = keystoreFile  
            storePassword =  
                providers.gradleProperty("networkpeer.keystore.password")  
            keyAlias = "networkpeer"  
            keyPassword = providers.gradleProperty("networkpeer.key.password")  
        }  
    }  
}  
  
buildTypes {  
    release {  
        signingConfig = signingConfigs.getByName("release")  
        isMinifyEnabled = true  
        proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),  
            "proguard-rules.pro")  
    }  
}
```


6.3 ProGuard/R8 Rules (proguard-rules.pro)

```
# AWS SDK v3
-keep class software.amazon.awssdk.** { *; }
-dontwarn software.amazon.awssdk.**

# Kotlinx Serialization
-keep class kotlinx.serialization.** { *; }

# Retrofit / OkHttp
-keep class retrofit2.** { *; }
-keep class okhttp3.** { *; }
-dontwarn okhttp3.**

# Socket.IO
-keep class io.socket.** { *; }

# Hilt / Dagger
-keep class dagger.** { *; }
-keep class javax.inject.** { *; }

# Room (if used)
-keep class androidx.room.** { *; }
```

6.4 Build AAB for Play Store

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/android
./gradlew bundleProductionRelease

# Output: app/build/outputs/bundle/productionRelease/app-production-release.aab
# Upload to Play Console → Release → Production
```

6.5 Play Console Setup

1. **Create app** → Package name: `com.networkpeer.mobile`
2. **App signing**: Use **Google Play App Signing** (upload `release.keystore` or let Google generate)

3. **Store listing:** Screenshots, feature graphic, privacy policy URL
 4. **Target API:** 34 (Android 14)
 5. **64-bit:** Enabled by default (arm64-v8a)
-

7. Testing

7.1 Unit Tests (JUnit + Kotlinx Serialization)

```
# All tests
./gradlew test

# Specific test class
./gradlew testDevelopmentDebugUnitTest --tests
    "com.networkpeer.mobile.core.model.NetworkPeerResponseModelsTest"

# Coverage
./gradlew jacocoTestReport
```

7.2 Contract Tests

```
// NetworkPeerResponseModelsTest.kt
@Test
fun `OTP requests use the Cognito challenge wire format`() {
    val result = json.decodeFromString<OtpRequestResult>("""
        {
            "challenge_id": "cognito-challenge",
            "expires_in_seconds": 600,
            "otp_length": 6,
            "delivery": {"transport": "sms", "to": "+15551234567"}
        }
    """)
    assertEquals("cognito-challenge", result.challengeId)
    assertEquals("sms", result.delivery.transport)
}
```

7.3 Instrumented Tests (AndroidJUnitRunner)

```
# Requires device/emulator
./gradlew connectedAndroidTest
```

8. CI/CD (GitHub Actions)

8.1 Workflow (`.github/workflows/android.yml`)

```

name: Android CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Cache Gradle
        uses: actions/cache@v4
        with:
          path: |
            ~/.gradle/caches
            ~/.gradle/wrapper
          key: ${{ runner.os }}-gradle-${{ hashFiles('**/*.gradle.kts', '**/gradle.properties') }}
      - name: Run tests
        run: ./gradlew test --no-daemon

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

```

```
- name: Build AAB
  run: ./gradlew bundleProductionRelease --no-daemon
- name: Upload AAB
  uses: actions/upload-artifact@v4
  with:
    name: app-aab
    path: app/build/outputs/bundle/productionRelease/*.aab
```

9. Troubleshooting

Issue	Solution
“SDK location not found”	Create <code>local.properties</code> with <code>sdk.dir</code> or set <code>ANDROID_HOME</code>
“Java not found”	Install JDK 17, set <code>JAVA_HOME</code>
“Could not resolve dependencies”	<code>./gradlew --refresh-dependencies</code>
“Cleartext traffic not permitted”	Add <code>network_security_config.xml</code> with local IP
FCM token not received	Check <code>google-services.json</code> in <code>app/</code> ; verify package name
ProGuard crashes	Add <code>-keep</code> rules for AWS SDK, Serialization, Retrofit
“INSTALL_FAILED_UPDATE_INCOMPATIBLE”	<code>adb uninstall com.networkpeer.mobile.debug</code> then reinstall

10. Quick Reference Commands

```
# Project root
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/android

# Build debug APK
./gradlew assembleDevelopmentDebug

# Install on connected device
./gradlew installDevelopmentDebug

# Run unit tests
./gradlew testDevelopmentDebugUnitTest

# Build release AAB
./gradlew bundleProductionRelease

# Clean
./gradlew clean

# Dependency graph
./gradlew dependencies --configuration developmentDebugRuntimeClasspath

# Lint
./gradlew lint

# Find Mac LAN IP for local backend
ifconfig | grep "inet " | grep -v 127.0.0.1
```

Last Updated: 2026-09-03 | NetworkPeer v1.3 | Kotlin 2.1 / Compose / Gradle 8.9 / AGP 8.9